

Diplomatura en programación web full stack con React JS

Módulo 4:

Introducción React, Next y Node

Unidad 2:

React/Next (parte 2)



Presentación

En esta unidad nos adentramos más en el mundo de Next incorporando los conocimientos necesarios para estructurar de forma correcta nuestros proyectos e incluir datos externos como contenido del mismo.



Objetivos

Que los participantes logren...

- Definir la estructura apropiada para su proyecto.
- Incorporen fuentes de datos externas.
- Manipulen los datos obtenidos para mostrarlos de la mejor manera.
- Aprendan los distintos tipos de renderizado.



Bloques temáticos

1. Carpetas, archivos y ruteo.
2. Data fetch y cache.
3. Listado y renderizado de elementos.
4. Tipos de renderizado.

1. Carpetas, archivos y ruteo

Next.js incluye por defecto un sistema de ruteo basado en archivos y carpetas mediante el cual es muy sencillo mantener la organización de nuestro código mientras definimos las rutas necesarias para nuestro proyecto.

Al inicializar nuestro proyecto encontramos dentro de la carpeta **app/src** que los archivos **layout.js** y **page.js** han sido creados de forma automática.

El archivo **layout.js** se utiliza para definir la apariencia que va a tener por fuera nuestra página, permitiéndonos de esta forma reutilizar la misma disposición de componentes en todas las páginas que lo requieran. Si vemos el archivo de ejemplo notaremos que incluye la estructura básica de un documento HTML e imprime la variable `children`, que es dónde **Next.js** pondrá el contenido de cada página que utilice este layout.

El archivo **page.js** se utiliza para definir el contenido de las páginas de nuestro sitio. Será siempre el último nodo en nuestro árbol de directorios y archivos al momento de definir las rutas. En este archivo debemos incluir todos los componentes necesarios para la ruta que hayamos definido.

Es importante aclarar que todo archivo **page.js** buscará hacia arriba en el árbol del proyecto el archivo **layout.js** más cercano a utilizar.

Para definir nuevas rutas bastará con crear una carpeta con el nombre de la ruta deseado e incluir un archivo **page.js** adentro. Así que si por ejemplo quisiéramos crear una URL para mostrar los servicios de nuestra empresa el archivo **page.js** correspondiente debería encontrarse en una ruta similar a esta:

src/app/servicios/page.js

De esta forma al acceder a la URL **<http://localhost:3000/servicios>** estaremos viendo el contenido de este archivo.

Las rutas pueden tener tantos niveles de anidación como sean necesarios brindando gran flexibilidad a la hora de organizarnos. En conjunto con los mecanismos descritos para el uso del archivo **layout.js**, permiten llevar adelante prácticamente cualquier tipo de ruteo que podamos necesitar.

2. Fetch y caché

Next.js extiende la API nativa de javascript para fetch agregando nuevas opciones y posibilidades, sobre todo para el manejo del cache de las peticiones que realicemos.

Por defecto (en la versión 15) ninguna petición realizada con fetch será almacenada en el caché por Next.js. Utilizando las opciones de cache y revalidate que Next.js permite pasar a fetch podemos controlar cuando se guardaran los datos y durante cuánto tiempo.

`fetch('https://...', { cache: 'force-cache' | 'no-store' })`

Si no especificamos una opción para cache Next.js realizará la petición en cada request mientras ejecutemos el proyecto en modo de desarrollo. Cuando realicemos un build se realizará la petición por única vez a menos que estemos implementando alguna característica de Next.js que fuerce al renderizado dinámico de las páginas (cookies, headers, searchParams, etc)

- **force-cache** busca una petición equivalente en el caché de datos y de encontrar una la devuelve. En caso de no encontrar una petición previa almacenada, realizará la petición y la almacenará en caché por tiempo indefinido.
- **no-store** evita que los datos se guarden en el caché de datos realizando la petición en cada request que se reciba.

`fetch('https://...', { next: { revalidate: false | 0 | number } })`

La opción de **revalidate** se utiliza en conjunto con la opción de cache que mencionamos anteriormente. Si no especificamos ningún valor para revalidate la información será almacenada por tiempo indefinido, de lo contrario podemos especificar un número de segundos luego del cual la información almacenada para esta petición será considerada vieja y se forzara a Next.js a realizar la petición nuevamente y volver a almacenar los resultados.

3. Listado y renderizado de elementos

Vamos a ver algunos patrones comunes a la hora de mostrar en nuestro proyecto datos que hayamos obtenido de alguna fuente externa. A modo de ejemplo vamos a usar la API de Rick and Morty (<https://rickandmortyapi.com/>) llamando al siguiente endpoint: **<https://rickandmortyapi.com/api/character>**. Vamos a asumir también a modo de ejemplo que tenemos ya armado un componente llamado Personaje que puede recibir como prop todos los datos de un personaje.

Map

Utilizamos el método map de un array permite ejecutar una función por cada elemento del array original y retornar un nuevo array con los resultados. En Next.js y React solemos utilizar map para, como en este caso, partiendo de un array de datos crudos, obtener un array de componentes que React pueda renderizar.

```
export default function Personaje(props) {  
  return (  
    <div>  
      <h2>{props.name}</h2>  
    </div>  
  )  
}
```

```
import Personaje from '@components/Personaje';

export default async function Personajes() {
  const response = await fetch('https://rickandmortyapi.com/api/character')

  const personajes = (await response.json()).results

  return (
    <div>
      <h1>Personajes</h1>
      {personajes.map((personaje) => (
        <Personaje key={personaje.id} {...personaje} />
      ))}
    </div>
  );
}
```

Render condicional

En las situaciones en las que necesitemos mostrar un contenido u otro basados en alguna condición podemos utilizar el renderizado condicional que no es más que un operador ternario (condicion ? verdadero : falso) aplicado a React. Como ejemplo vamos a aplicar el renderizado condicional para mostrar un emoji indicando el estado de cada personaje:

```
export default function Personaje(props) {  
  return (  
    <div>  
      <h2>{props.name}</h2>  
      {props.status === 'Alive' ? <p>👍</p> : <p>👎</p>}  
    </div>  
  )  
}
```

Filter

Otro patrón común que podemos encontrarnos es el del filtrado. Mediante el método filter podemos pasar cada elemento de un array por una función que determina devolviendo true o false si este elemento formará parte del nuevo array. En el siguiente ejemplo vemos cómo podemos utilizar filter encadenado con map para listar solo los personajes que sean humanos:

```
import Personaje from '@components/Personaje';

export default async function Personajes() {
  const response = await fetch('https://rickandmortyapi.com/api/character')
  const personajes = (await response.json()).results

  return (
    <div>
      <h1>Personajes</h1>
      {personajes.filter(personaje => personaje.species ===
'Human').map((personaje) => (
        <Personaje key={personaje.id} {...personaje} />
      ))}
    </div>
  );
}
```

4. Tipos de renderizado (estático, dinámico)

Por defecto Next.js intentará renderizar todas las rutas de páginas como estáticas. Esto quiere decir que Next.js va a utilizar el cache al momento de realizar el build y almacenará las peticiones y resultados de pre renderizar las páginas para brindar al usuario una experiencia de navegación más rápida. Esta opción es ideal para la gran mayoría de sitios orientados al contenido y que no se actualizan muy frecuentemente porque ahorra peticiones innecesarias. Cabe aclarar que este modo de renderizado funciona a la par de los mecanismos de cache que ya vimos.

En algunos casos podemos necesitar que una ruta se renderice de forma dinámica, ya sea porque necesitamos que el contenido esté siempre actualizado o porque debemos adaptar el contenido de la página al usuario que la esté visitando. En estos casos podemos exportar dentro del código de la ruta en cuestión la variable `dynamic` con su valor:

```
export default function Pagina() {  
  // ... codigo de la pagina  
}  
  
export dynamic = 'force-dynamic'
```

Conclusión

En resumen, aprendimos cómo estructurar nuestro proyecto y sus archivos para armar las rutas necesarias. Incorporar datos externos mediante el uso de fetch y los mostramos y manipulamos para su presentación al usuario.

En la próxima unidad veremos...

Nos introduciremos en el mundo de Node, la forma de usar JavaScript del lado del servidor. Veremos cómo instalarlo y dar nuestros primeros pasos.



Bibliografía utilizada y sugerida

Libros y otros manuscritos:

Alex Banks & Eve Porcello. Learning React: desarrollo web funcional con React y Redux. 2017.

Artículos de revista en formato electrónico:

Enrutando-en-react. Disponible desde la URL:

<https://medium.com/@simonhoyos/enrutando-en-react-cd9e4ad6e3d3>

MDN Web Docs. Disponible desde la URL: <https://developer.mozilla.org/>

React. Disponible desde la URL: <https://es.reactjs.org/>

Next. Disponible desde la URL: <https://nextjs.org/>